

Università degli Studi di Parma
Corso di Laurea Magistrale in Ingegneria Informatica
Curriculum: **Intelligenza Artificiale**

Image Captioning con CNN–LSTM su Flickr8k

Obiettivo: Il presente elaborato analizza e spiega in modo approfondito l'assegnamento n. 27 del corso di *Deep Learning e Modelli Generativi*, che richiede lo sviluppo di un sistema di *image captioning* capace di generare automaticamente descrizioni testuali a partire da immagini di input. Il modello proposto si basa su un'architettura ibrida **CNN–LSTM**, in cui la rete convoluzionale estrae le caratteristiche visive e la rete LSTM produce la sequenza linguistica corrispondente. L'intero progetto è stato realizzato in **PyTorch**, con l'obiettivo di costruire un sistema completo, chiaro e replicabile per lo studio e la comprensione del processo di generazione automatica di didascalie.

Studente: Giuseppe Dimonte
Docente: Prof. Andrea Prati
Corso: Deep Learning e Modelli Generativi

Sommario

Il presente elaborato descrive lo sviluppo di un sistema di *image captioning*, il cui obiettivo è generare automaticamente una descrizione testuale coerente e significativa a partire da un'immagine di input. Il lavoro si basa sull'assegnamento n. 27 del corso di *Deep Learning e Modelli Generativi* e utilizza il dataset pubblico **Flickr8k**, costituito da 8.000 immagini, ognuna associata a cinque didascalie scritte da persone diverse.

L'architettura implementata combina una rete neurale convoluzionale (CNN) per l'estrazione delle caratteristiche visive con una rete LSTM (*Long Short-Term Memory*) per la generazione sequenziale delle parole, secondo il paradigma *image-to-sequence*. Durante l'addestramento sono state adottate tecniche di ottimizzazione avanzate, come *teacher forcing*, *scheduled sampling*, *label smoothing* e *gradient clipping*, al fine di migliorare la stabilità e la qualità delle didascalie generate.

Il documento illustra in modo dettagliato tutte le fasi del progetto — dalla pulizia e codifica dei dati, alla costruzione del modello e alla valutazione tramite la metrica BLEU — con l'obiettivo di fornire una visione completa, didattica e replicabile del processo di generazione automatica di descrizioni visive.

Capitolo 1

Introduzione

1.1 Perché l’image captioning?

Immaginate di mostrare una fotografia a un assistente digitale e ricevere immediatamente una descrizione testuale chiara, corretta e coerente: questo è l’obiettivo dell’*image captioning*. Si tratta di una delle applicazioni più rappresentative dell’intelligenza artificiale moderna, poiché unisce la **comprensione visiva** (tipica delle reti convoluzionali) alla **generazione linguistica** (tipica dei modelli sequenziali o di linguaggio).

L’importanza di questa disciplina è molteplice:

- **Accessibilità:** un sistema in grado di descrivere automaticamente immagini può aiutare persone ipovedenti a comprendere il contenuto visivo di fotografie, video o pagine web.
- **Organizzazione dei contenuti:** nelle grandi piattaforme di immagini (come Flickr, Google Photos o Instagram), generare descrizioni automatiche aiuta a migliorare la ricerca e l’indicizzazione.
- **Sistemi multimodali:** l’image captioning è una componente essenziale in sistemi che combinano visione e linguaggio, come i modelli *Vision-Language*, i chatbot visivi o i generatori di testo condizionati da immagini.

Inoltre, l’image captioning rappresenta un eccellente banco di prova per il **deep learning multimodale**, perché obbliga il modello a combinare due forme di informazione profondamente diverse: i pixel e le parole. Questo comporta non solo un problema di apprendimento supervisionato, ma anche una sfida cognitiva: tradurre il linguaggio visivo in linguaggio naturale.

Messaggio chiave

L’image captioning è il punto d’incontro tra la visione artificiale e l’elaborazione del linguaggio naturale: un modello che “vede” come un umano e “parla” come un umano.

1.2 Contesto del progetto

Il lavoro qui presentato si inserisce nell’ambito del corso di *Deep Learning e Modelli Generativi*, ed è relativo all’assegnamento n.27. L’obiettivo dell’assegnamento è lo sviluppo, in **PyTorch**,

di un sistema completo di image captioning capace di:

1. estrarre le caratteristiche visive salienti da un'immagine tramite una rete neurale convoluzionale (CNN);
2. utilizzare tali informazioni come contesto per una rete LSTM (Long Short-Term Memory), che genera una frase parola per parola;
3. decodificare la sequenza di parole in linguaggio naturale grazie a un classificatore finale (MLP).

Il dataset utilizzato è **Flickr8k**, una raccolta di 8.000 immagini annotate con cinque descrizioni testuali ciascuna. Tale dataset è spesso impiegato come benchmark per modelli di captioning di piccola scala, poiché consente di sperimentare con architetture relativamente leggere ma efficaci.

Il progetto richiede non solo di implementare la pipeline di addestramento e valutazione, ma anche di comprendere a fondo le scelte architetturali, la gestione dei dati, le metriche di qualità e le strategie di ottimizzazione.

1.3 Idea di base

La pipeline alla base del sistema può essere riassunta in tre passaggi fondamentali:

1. **Feature extraction:** una **CNN**, pre-addestrata su un grande dataset come ImageNet, trasforma l'immagine in un vettore di caratteristiche numeriche (*feature vector*) che rappresentano il contenuto visivo in forma compatta.
2. **Sequenza linguistica:** una **LSTM** riceve questo vettore come stato iniziale e genera la didascalia parola per parola, apprendendo la correlazione tra caratteristiche visive e parole del linguaggio naturale.
3. **Decodifica finale:** un piccolo **MLP** (Multi-Layer Perceptron) trasforma lo stato nascosto della LSTM in una distribuzione di probabilità sulle parole del vocabolario, scegliendo quella più probabile a ogni passo.

Messaggio chiave

Le CNN “vedono”, le LSTM “parlano”. La CNN comprime il contenuto visivo in un vettore, la LSTM lo utilizza per costruire una frase coerente e plausibile che descriva ciò che l'immagine rappresenta.

1.4 Obiettivi didattici

Oltre all'aspetto implementativo, questo progetto ha anche una forte valenza formativa:

- comprendere come modelli differenti (CNN e LSTM) possano essere integrati in un'unica pipeline;

- imparare a gestire dataset multimodali (immagini + testo);
- applicare tecniche di pre-processing e tokenizzazione del linguaggio;
- sperimentare metodi di ottimizzazione e regolarizzazione tipici del deep learning moderno;
- interpretare le metriche di valutazione come BLEU e analizzare qualitativamente le caption generate.

Questo elaborato, pertanto, non si limita a presentare un modello funzionante, ma mira a fornire una spiegazione esaustiva, passo per passo, dell'intero processo di costruzione di un sistema di image captioning moderno e riproducibile.

Capitolo 2

Il Dataset Flickr8k

2.1 Introduzione al dataset

Per addestrare un modello di *image captioning* è necessario disporre di un dataset che colleghi il dominio visivo (le immagini) a quello linguistico (le descrizioni testuali). Il dataset scelto per questo progetto è **Flickr8k**, una raccolta pubblica ampiamente utilizzata nella ricerca accademica per modelli di generazione automatica di didascalie.

Il dataset, originariamente sviluppato presso la University of Illinois, contiene:

- **8.000 immagini** di scene naturali (persone, animali, oggetti, attività);
- **5 descrizioni testuali per ciascuna immagine**, scritte manualmente da annotatori umani;
- una suddivisione ufficiale in *training set*, *validation set* e *test set*.

Le immagini provengono dal sito web *Flickr.com* e rappresentano situazioni della vita quotidiana: bambini che giocano, animali in movimento, persone impegnate in attività sportive o ricreative. Le descrizioni testuali, o *caption*, offrono un breve riassunto del contenuto visivo espresso in linguaggio naturale. Ad esempio:

Esempio di didascalia

Immagine: un cane che corre su un prato.

Didascalie associate:

- A brown dog is running through the grass.
- The dog runs across a grassy field.
- A dog is playing outside on the grass.
- The brown dog is in motion.
- A dog runs through a green field.

Come si può osservare, le descrizioni non sono identiche ma esprimono lo stesso concetto con parole diverse. Questo aspetto è cruciale, poiché durante l'addestramento il modello apprende

non solo la relazione tra immagine e testo, ma anche la varietà linguistica con cui è possibile descrivere la stessa scena.

2.2 Struttura dei file

Dopo aver scaricato il dataset (ad esempio dal repository GitHub [goodwillyoga/Flickr8k_dataset](https://github.com/goodwillyoga/Flickr8k_dataset)), la struttura delle cartelle appare come segue:

```
Flickr8k_Dataset/
|-- Flickr8k_Dataset/          # Contiene tutte le immagini (.jpg)
|-- Flickr8k_text/
|   |-- Flickr8k.token.txt     # Tutte le descrizioni
|   |-- Flickr_8k.trainImages.txt
|   |-- Flickr_8k.testImages.txt
|   \-- Flickr_8k.devImages.txt
```

Il file `Flickr8k.token.txt` contiene tutte le associazioni tra immagini e descrizioni. Ogni riga segue il formato:

```
1000268201_693b08cb0e.jpg#0  A child in a pink dress is climbing up a set of stairs.
1000268201_693b08cb0e.jpg#1  A girl going into a wooden building.
```

Dove:

- la parte prima del simbolo “#” indica il nome dell’immagine;
- il numero dopo “#” rappresenta l’indice progressivo della descrizione (da 0 a 4);
- dopo uno spazio segue la didascalia vera e propria.

2.3 Pulizia del testo e pre-processing

Prima di poter addestrare una rete neurale, le frasi devono essere trasformate in una rappresentazione numerica. Il primo passo consiste nella **pulizia del testo**, che serve a uniformare le descrizioni e ridurre la variabilità inutile.

Le principali operazioni di pulizia sono:

- conversione di tutte le parole in minuscolo;
- rimozione di numeri e punteggiatura;
- eliminazione di parole composte da un solo carattere (es. “a”, “s”);
- rimozione di spazi multipli o finali.

Listing 2.1: Funzione di pulizia del testo delle caption.

```
1 import re
2
3 def clean_caption(text):
4     """
5     Pulisce una didascalia secondo le regole del progetto.
6     """
7     text = text.lower() # converte tutto in
8                          # minuscolo
9     text = re.sub(r"[^a-z\\s]", "", text) # rimuove numeri e
10                                           # punteggiatura
11     words = [w for w in text.split() if len(w) > 1] # elimina parole di
12                                                       # 1 carattere
13     return " ".join(words)
```

Spiegazione

- `lower()` converte l'intera frase in minuscolo per evitare duplicazioni come "Dog" e "dog".
- `re.sub(...)` rimuove simboli, cifre e punteggiatura.
- La list comprehension elimina parole corte e non significative.
- Infine, si ricostruisce la frase pulita.

Dopo la pulizia, si crea un dizionario Python che associa a ogni immagine le sue cinque descrizioni pulite:

Listing 2.2: Creazione del dizionario delle caption.

```
1 captions_dict = {}
2
3 for line in open("Flickr8k_text/Flickr8k.token.txt"):
4     image_id, caption = line.split("\t")
5     image_id = image_id.split("#")[0]
6     caption = clean_caption(caption)
7     captions_dict.setdefault(image_id, []).append(caption)
```

Struttura del dizionario

Ogni chiave del dizionario (`image_id`) corrisponde al nome di un file immagine, e il valore associato è una lista di cinque frasi pulite. Questo formato verrà utilizzato successivamente per creare il dataset PyTorch personalizzato.

2.4 Costruzione del vocabolario

Dopo la pulizia, le parole devono essere convertite in numeri. Ogni parola del dataset viene quindi mappata a un indice intero all'interno di un **vocabolario**. Il vocabolario è costruito

includendo solo le parole che compaiono almeno un certo numero di volte (ad esempio 5), per ridurre la dimensione del dizionario e migliorare la generalizzazione.

Vengono aggiunti inoltre quattro token speciali:

- `<bos>` — inizio della frase;
- `<eos>` — fine della frase;
- `<pad>` — padding per uniformare la lunghezza delle sequenze;
- `<unk>` — parole non presenti nel vocabolario.

Listing 2.3: Costruzione del vocabolario.

```
1 from collections import Counter
2
3 counter = Counter()
4 for caps in captions_dict.values():
5     for c in caps:
6         counter.update(c.split())
7
8 vocab = [word for word, count in counter.items() if count >= 5]
9 vocab = ["<pad>", "<bos>", "<eos>", "<unk>"] + sorted(vocab)
10 word2idx = {w: i for i, w in enumerate(vocab)}
11 idx2word = {i: w for w, i in word2idx.items()}
```

Nota

Il dizionario `word2idx` verrà salvato in un file (ad esempio `vocab.json`) per essere caricato durante l'addestramento. Impostare una soglia minima di frequenza (come 5 occorrenze) aiuta a ridurre la complessità del modello e a limitare l'overfitting su parole rare.

2.5 Riepilogo

In questo capitolo abbiamo:

- esplorato la struttura del dataset `Flickr8k`;
- compreso il formato dei file e delle descrizioni;
- pulito e normalizzato il testo delle caption;
- costruito un dizionario e un vocabolario numerico.

Questi passaggi costituiscono la base per la fase successiva: la creazione del **dataset personalizzato PyTorch**, che collegherà in modo efficiente le immagini preprocessate e le didascalie tokenizzate.

Riferimento al codice

Nel repository finale, la preparazione dei dati è stata consolidata negli script `src/vocab_builder.py` e `src/prepare_data.py`, utilizzati per generare il vocabolario e gli split effettivamente impiegati negli esperimenti.

Capitolo 3

Fondamenti di Deep Learning

3.1 Dai neuroni artificiali alle reti profonde

Il *deep learning* è un ramo dell'apprendimento automatico che utilizza reti neurali multistrato per apprendere relazioni complesse a partire dai dati. Il blocco fondamentale di una rete neurale è il **neurone artificiale**, che rappresenta una versione semplificata del neurone biologico.

Ogni neurone riceve uno o più input numerici, li moltiplica per pesi associati, aggiunge un termine di bias e applica una funzione di attivazione non lineare:

$$y = f(\mathbf{w}^\top \mathbf{x} + b) \quad (3.1)$$

dove:

- $\mathbf{x}$ è il vettore degli input;
- $\mathbf{w}$ rappresenta i pesi appresi durante l'addestramento;
- b è il bias, un termine aggiuntivo per spostare la funzione di attivazione;
- $f(\cdot)$ è la funzione di attivazione (ad esempio ReLU o sigmoid).

La funzione di attivazione introduce una componente non lineare nel modello, permettendo alla rete di rappresentare relazioni complesse che non possono essere catturate da modelli puramente lineari.

Idea chiave

Senza funzioni di attivazione non lineari, anche più strati di neuroni si ridurrebbero a un'unica trasformazione lineare: la rete non sarebbe in grado di apprendere relazioni complesse tra i dati.

3.2 Architettura di una rete neurale

Una rete neurale profonda (DNN, *Deep Neural Network*) è costruita impilando più strati di neuroni, in modo che ciascun livello apprenda una rappresentazione progressivamente più astratta dei dati.

- **Strato di input:** riceve i dati grezzi (ad esempio i pixel di un'immagine o le parole codificate come numeri).
- **Strati nascosti:** elaborano rappresentazioni intermedie tramite moltiplicazioni di matrici e funzioni di attivazione.
- **Strato di output:** produce il risultato finale, come una classe o una sequenza di parole.

Un esempio minimale di rete neurale in PyTorch può essere scritto così:

Listing 3.1: Esempio di rete neurale feed-forward in PyTorch.

```

1  import torch
2  import torch.nn as nn
3
4  class SimpleNet(nn.Module):
5      def __init__(self, input_dim, hidden_dim, output_dim):
6          super(SimpleNet, self).__init__()
7          self.fc1 = nn.Linear(input_dim, hidden_dim)
8          self.relu = nn.ReLU()
9          self.fc2 = nn.Linear(hidden_dim, output_dim)
10
11     def forward(self, x):
12         x = self.relu(self.fc1(x))
13         return self.fc2(x)

```

Spiegazione

Questa piccola rete è composta da due strati lineari separati da una funzione di attivazione ReLU. Durante l'addestramento, la rete apprende i valori dei pesi all'interno di `fc1` e `fc2` per minimizzare l'errore di predizione.

3.3 Il processo di addestramento

L'obiettivo dell'addestramento di una rete neurale è trovare l'insieme di pesi che minimizza una data **funzione di perdita** (*loss function*) — cioè una misura dell'errore tra le predizioni del modello e i valori corretti.

Il processo tipico di addestramento segue questi passaggi:

1. Eseguire una **forward pass** per calcolare le predizioni.
2. Calcolare la **funzione di perdita** (es. entropia incrociata o errore quadratico medio).
3. Calcolare il gradiente della perdita rispetto ai pesi tramite la **backpropagation**.
4. Aggiornare i pesi usando un algoritmo di ottimizzazione come **Stochastic Gradient Descent (SGD)** o **Adam**.

In PyTorch, un singolo passo di ottimizzazione può essere riassunto così:

Listing 3.2: Esempio di passo di ottimizzazione in PyTorch.

```

1 optimizer.zero_grad()      # azzera i gradienti accumulati
2 outputs = model(inputs)    # forward pass
3 loss = criterion(outputs, targets) # calcola la loss
4 loss.backward()            # backpropagation
5 optimizer.step()           # aggiorna i pesi

```

Concetto: discesa del gradiente

La discesa del gradiente aggiorna ciascun peso nella direzione che riduce la perdita:

$$w := w - \eta \frac{\partial L}{\partial w}$$

dove η è il tasso di apprendimento (*learning rate*), un iperparametro che controlla l'ampiezza del passo di aggiornamento.

3.4 Epoche, batch e iterazioni

L'addestramento delle reti neurali avviene tipicamente in più cicli chiamati **epoche**. Un'epoca corrisponde al momento in cui il modello ha visto l'intero dataset una volta. Poiché elaborare tutto il dataset in un unico blocco sarebbe inefficiente, si divide il set di dati in sottoinsiemi più piccoli chiamati **batch**.

- Ogni **batch** contiene un piccolo gruppo di esempi (ad esempio 32 o 64).
- Una **iterazione** corrisponde a un aggiornamento dei pesi basato su un singolo batch.
- Dopo aver elaborato tutti i batch, si completa un'epoca.

Esempio

Se il dataset contiene 8.000 campioni e la dimensione del batch è 64, allora ogni epoca è composta da $\frac{8000}{64} \approx 125$ iterazioni.

Capire la relazione tra epoche, batch e iterazioni è fondamentale per regolare l'addestramento e prevenire problemi come *overfitting* o *underfitting*.

3.5 Overfitting e regolarizzazione

Un problema comune nel deep learning è l'**overfitting**, che si verifica quando il modello impara troppo bene i dati di addestramento ma fallisce nel generalizzare su dati nuovi. In pratica, il modello “memorizza” anziché “apprendere”.

Per contrastarlo, si utilizzano varie tecniche di **regolarizzazione**:

- **Dropout**: disattiva casualmente alcuni neuroni durante l'addestramento per evitare co-adattamenti.

- **Weight decay:** penalizza pesi troppo grandi mediante una regolarizzazione L_2 .
- **Early stopping:** interrompe l'addestramento quando la perdita di validazione smette di migliorare.
- **Data augmentation:** aumenta la varietà dei dati applicando trasformazioni alle immagini (rotazioni, tagli, flip, ecc.).

Nota pratica

Nel progetto Flickr8k, tecniche come *dropout* ed *early stopping* vengono applicate per migliorare la capacità di generalizzazione del modello CNN-LSTM.

3.6 Funzioni di perdita e algoritmi di ottimizzazione

La scelta della funzione di perdita dipende dal tipo di problema:

- **Cross-Entropy Loss:** usata per classificazione o generazione di sequenze (come la caption di un'immagine).
- **Mean Squared Error (MSE):** usata per problemi di regressione.

Gli algoritmi di ottimizzazione più diffusi includono:

- **SGD (Stochastic Gradient Descent):** semplice ed efficace, aggiorna i pesi a piccoli passi.
- **Adam:** versione adattiva dell'SGD, adatta dinamicamente il learning rate per ogni parametro.
- **RMSProp:** simile ad Adam, particolarmente efficace per reti ricorrenti.

Listing 3.3: Definizione della funzione di perdita e dell'ottimizzatore in PyTorch.

```
1 criterion = nn.CrossEntropyLoss()
2 optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
```

3.7 Riepilogo

In questo capitolo sono stati introdotti i concetti fondamentali del deep learning che costituiscono la base teorica del modello CNN-LSTM sviluppato nei capitoli successivi:

- la struttura delle reti neurali e il ruolo delle funzioni di attivazione;
- il ciclo di addestramento, la backpropagation e l'ottimizzazione;
- il significato di epoca, batch e iterazione;
- le principali tecniche di regolarizzazione per migliorare la generalizzazione.

Queste basi teoriche sono indispensabili per comprendere come nei prossimi capitoli — dedicati a CNN e LSTM — i due mondi della visione artificiale e del linguaggio naturale vengano integrati in un unico modello coerente.

Riferimento al codice

I frammenti di codice di esempio di questo capitolo hanno valore illustrativo; nel repository finale, i file realmente coinvolti nella pipeline sono `src/dataset.py`, `src/model.py`, `src/train.py`, `src/eval.py` e `src/visualize.py`.

Capitolo 4

Reti Neurali Convoluzionali (CNN)

4.1 Introduzione alle CNN

Le **reti neurali convoluzionali** (*Convolutional Neural Networks*, CNN) rappresentano il cuore della moderna visione artificiale. Sono progettate per estrarre automaticamente caratteristiche visive (*features*) da immagini, riconoscendo pattern spaziali come bordi, forme, colori e oggetti.

A differenza delle reti completamente connesse, dove ogni neurone è collegato a tutti gli input, una CNN sfrutta la struttura locale dei dati visivi: un pixel è influenzato principalmente dai pixel vicini. Questo principio riduce il numero di parametri e migliora l'efficienza.

Idea chiave

Le CNN imparano a “vedere” come l'occhio umano: nei primi strati riconoscono bordi e texture, negli strati più profondi identificano oggetti complessi e concetti semantici.

4.2 Operazione di convoluzione

L'operazione fondamentale di una CNN è la **convoluzione**, che consiste nel far scorrere un piccolo filtro (detto anche *kernel*) su tutta l'immagine per estrarne caratteristiche locali.

Matematicamente, per un'immagine I e un filtro K di dimensione $m \times n$, la convoluzione è definita come:

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) \cdot K(m, n)$$

Il risultato, chiamato **feature map**, indica quanto un certo pattern (il filtro) è presente in ogni regione dell'immagine.

In PyTorch, un livello convoluzionale può essere definito in modo molto semplice:

Listing 4.1: Esempio di layer convoluzionale in PyTorch.

```
1 import torch
2 import torch.nn as nn
3
4 conv = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3, stride
    =1, padding=1)
```

```

5 x = torch.randn(1, 3, 224, 224) # batch=1, canali=3, immagine 224x224
6 y = conv(x)
7 print(y.shape) # -> torch.Size([1, 16, 224, 224])

```

Spiegazione

Il layer convoluzionale riceve un'immagine RGB (3 canali) e applica 16 filtri 3×3 . Il risultato è una rappresentazione dell'immagine con 16 canali, ognuno dei quali evidenzia una diversa caratteristica visiva appresa.

4.3 Pooling e riduzione dimensionale

Dopo la convoluzione, è comune applicare un'operazione di **pooling**, che riduce la dimensione spaziale delle feature map mantenendo le informazioni più rilevanti. Il tipo più usato è il **max pooling**, che conserva solo il valore massimo all'interno di una finestra locale.

Listing 4.2: Esempio di operazione di max pooling.

```

1 pool = nn.MaxPool2d(kernel_size=2, stride=2)
2 out = pool(y)
3 print(out.shape) # -> torch.Size([1, 16, 112, 112])

```

Effetto del pooling

La risoluzione spaziale è dimezzata (da 224x224 a 112x112), ma la rete conserva le caratteristiche più significative. Questo migliora la robustezza e riduce il numero di parametri.

4.4 Funzioni di attivazione nelle CNN

Ogni strato convoluzionale è seguito da una funzione di attivazione non lineare, solitamente la **ReLU (Rectified Linear Unit)**:

$$f(x) = \max(0, x)$$

La ReLU accelera l'addestramento e riduce il problema del gradiente che svanisce. In PyTorch, viene integrata direttamente nel flusso della rete:

Listing 4.3: Uso della funzione ReLU in una CNN.

```

1 model = nn.Sequential(
2     nn.Conv2d(3, 16, 3, 1, 1),
3     nn.ReLU(),
4     nn.MaxPool2d(2, 2)
5 )

```

4.5 Struttura generale di una CNN

Una CNN completa combina più livelli convoluzionali, di pooling e non linearità. Negli ultimi strati, le feature vengono “appiattite” in un vettore e passate a uno o più strati lineari (*fully connected*) per la classificazione o altre predizioni.

Listing 4.4: Esempio di CNN semplificata.

```
1 class SimpleCNN(nn.Module):
2     def __init__(self):
3         super(SimpleCNN, self).__init__()
4         self.features = nn.Sequential(
5             nn.Conv2d(3, 32, 3, 1, 1),
6             nn.ReLU(),
7             nn.MaxPool2d(2),
8             nn.Conv2d(32, 64, 3, 1, 1),
9             nn.ReLU(),
10            nn.MaxPool2d(2)
11        )
12        self.classifier = nn.Sequential(
13            nn.Linear(64 * 56 * 56, 256),
14            nn.ReLU(),
15            nn.Linear(256, 10)
16        )
17
18    def forward(self, x):
19        x = self.features(x)
20        x = x.view(x.size(0), -1) # flatten
21        return self.classifier(x)
```

Nota

In questa rete, due blocchi di convoluzione+pooling estraggono caratteristiche visive, mentre la sezione `classifier` effettua la previsione finale. Nel caso dell’image captioning, invece della classificazione, le feature estratte vengono passate a un modulo LSTM.

4.6 Transfer learning e feature extraction

Addestrare una CNN da zero richiede milioni di immagini e molto tempo. Per questo motivo, si usa spesso il **transfer learning**: si riutilizzano i pesi di una rete già addestrata (ad esempio su ImageNet) e la si adatta al nuovo compito.

In questo progetto, la CNN non viene riaddestrata interamente: si usa un modello pre-addestrato come estrattore di feature, “congelando” i suoi pesi.

Listing 4.5: Uso di InceptionV3 come estrattore di feature.

```
1 from torchvision import models
2
3 cnn = models.inception_v3(pretrained=True)
```

```
4 for param in cnn.parameters():
5     param.requires_grad = False # congela i pesi
6
7 # Rimuove l'ultimo livello di classificazione
8 feature_extractor = nn.Sequential(*list(cnn.children())[:-1])
```

Applicazione nel progetto

Le feature estratte dall'ultima parte della CNN (di solito un vettore di 2048 dimensioni) vengono fornite come input iniziale alla LSTM. In questo modo, la LSTM può “parlare” partendo da una rappresentazione visiva dell'immagine.

4.7 Riepilogo

In questo capitolo abbiamo:

- compreso il principio della convoluzione e del pooling;
- visto come le CNN estraggono caratteristiche gerarchiche dalle immagini;
- introdotto il concetto di transfer learning e l'uso di reti pre-addestrate;
- collegato la fase di estrazione visiva alla generazione testuale della LSTM.

Riferimento al codice

Nel repository finale, l'encoder convoluzionale e il decoder linguistico sono implementati congiuntamente nel file `src/model.py`, che definisce la componente CNN basata su InceptionV3 e il decoder LSTM per la generazione delle didascalie.

Capitolo 5

Reti LSTM e Generazione di Sequenze

5.1 Introduzione alle reti ricorrenti

Quando l'obiettivo è generare o comprendere una sequenza di parole, entra in gioco una classe speciale di reti neurali: le **reti ricorrenti** (*Recurrent Neural Networks*, RNN). A differenza delle CNN, che analizzano informazioni spaziali, le RNN gestiscono informazioni *temporali* o sequenziali, mantenendo in memoria ciò che è stato elaborato in precedenza.

Tuttavia, le RNN classiche soffrono di due problemi fondamentali:

- **Vanishing gradient:** il gradiente diventa troppo piccolo durante la backpropagation, impedendo l'apprendimento a lungo termine.
- **Exploding gradient:** il gradiente cresce in modo incontrollato, destabilizzando l'addestramento.

Per risolvere questi limiti, sono state introdotte le **Long Short-Term Memory networks (LSTM)**, una variante più stabile e potente delle RNN standard.

Idea chiave

Le LSTM “ricordano” informazioni rilevanti per lunghi intervalli temporali e “dimenticano” quelle inutili, gestendo la memoria interna attraverso porte di controllo.

5.2 Struttura interna di una LSTM

Ogni unità LSTM contiene tre **porte** principali:

- la **porta di input** (i_t), che decide quante nuove informazioni aggiungere alla memoria;
- la **porta di dimenticanza** (f_t), che regola quante informazioni precedenti scartare;
- la **porta di output** (o_t), che determina quanta parte dello stato interno viene usata come output.

Le equazioni di una cella LSTM, per il passo temporale t , sono le seguenti:

$$\begin{aligned}f_t &= \sigma(W_f[h_{t-1}, x_t] + b_f) \\i_t &= \sigma(W_i[h_{t-1}, x_t] + b_i) \\\tilde{C}_t &= \tanh(W_C[h_{t-1}, x_t] + b_C) \\C_t &= f_t * C_{t-1} + i_t * \tilde{C}_t \\o_t &= \sigma(W_o[h_{t-1}, x_t] + b_o) \\h_t &= o_t * \tanh(C_t)\end{aligned}$$

dove:

- x_t è l'input corrente (ad esempio la parola al tempo t);
- h_t è lo stato nascosto (output della cella);
- C_t è lo stato di memoria;
- σ è la funzione sigmoide;
- $\tanh$ è la funzione tangente iperbolica.

Interpretazione intuitiva

La LSTM decide a ogni passo “quanto ricordare” e “quanto dimenticare”. Questo meccanismo la rende ideale per processi sequenziali come la generazione di testo o la traduzione automatica.

5.3 Generazione di sequenze di parole

Nel contesto dell'*image captioning*, la LSTM riceve in ingresso un vettore di caratteristiche estratto dalla CNN (una rappresentazione compatta dell'immagine) e genera una sequenza di parole una alla volta.

- La prima parola generata è solitamente `<bos>` (“begin of sentence”).
- A ogni passo successivo, la LSTM riceve come input la parola precedente (convertita in embedding numerico).
- Il processo continua fino a generare il token `<eos>` (“end of sentence”).

Il meccanismo è illustrato nel seguente schema:

feature image $\rightarrow$ LSTM $\rightarrow$ caption: “a dog runs on the grass”

Listing 5.1: Esempio semplificato di decodifica con LSTM.

```
1 import torch
2 import torch.nn as nn
3
```

```

4 class CaptionLSTM(nn.Module):
5     def __init__(self, embed_dim, hidden_dim, vocab_size):
6         super(CaptionLSTM, self).__init__()
7         self.embedding = nn.Embedding(vocab_size, embed_dim)
8         self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True)
9         self.fc = nn.Linear(hidden_dim, vocab_size)
10
11     def forward(self, captions, features):
12         embeddings = self.embedding(captions)
13         inputs = torch.cat((features.unsqueeze(1), embeddings), 1)
14         outputs, _ = self.lstm(inputs)
15         return self.fc(outputs)

```

Spiegazione passo-passo

- **embedding**: trasforma gli indici delle parole in vettori densi (embedding).
- **features**: rappresenta l'immagine compressa dalla CNN.
- **cat(...)**: concatena le feature iniziali all'inizio della sequenza di parole.
- **fc**: proietta lo stato nascosto in probabilità sulle parole del vocabolario.

5.4 Addestramento e teacher forcing

Durante l'addestramento, la LSTM utilizza un approccio chiamato **teacher forcing**: a ogni passo temporale riceve la parola corretta del dataset come input successivo, invece di quella che ha predetto. Questo accelera la convergenza, ma può rendere il modello meno robusto in fase di test.

Per mitigare il problema, si applica lo **scheduled sampling**, che riduce progressivamente il teacher forcing: con il passare delle epoche, il modello utilizza sempre più spesso le proprie predizioni come input.

Listing 5.2: Scheduled sampling nel training loop.

```

1 teacher_forcing_ratio = max(0.3, 0.7 - epoch / total_epochs * 0.4)
2 if random.random() < teacher_forcing_ratio:
3     next_input = target_token # usa la parola corretta
4 else:
5     next_input = predicted_token # usa la parola predetta

```

Nota

Questo meccanismo rende la rete più stabile e capace di generare frasi realistiche anche quando non ha accesso al ground truth.

5.5 Funzione di perdita e mascheramento

Per addestrare correttamente la LSTM, si utilizza la **Cross-Entropy Loss**, che misura la distanza tra le parole predette e quelle corrette. Poiché le sequenze hanno lunghezze diverse, si applica un **masking** per ignorare i token di padding (<pad>).

Listing 5.3: Calcolo della loss con mascheramento.

```
1 criterion = nn.CrossEntropyLoss(ignore_index=pad_idx)
2 loss = criterion(predictions.view(-1, vocab_size), targets.view(-1))
```

5.6 Decodifica in fase di test

Durante la generazione (fase di test), la LSTM non riceve più parole corrette ma deve generare una didascalia partendo solo dalle feature dell'immagine. Si parte dal token <bos>, e a ogni passo si sceglie la parola più probabile:

Listing 5.4: Decodifica greedy.

```
1 word = "<bos>"
2 caption = []
3 for _ in range(max_len):
4     input_word = torch.tensor([[word2idx[word]]])
5     output, (h, c) = lstm(input_word, (h, c))
6     pred = fc(output.squeeze(1))
7     word = idx2word[pred.argmax(dim=1).item()]
8     if word == "<eos>":
9         break
10    caption.append(word)
```

Nota

L'approccio mostrato è detto **greedy decoding**: a ogni passo si sceglie la parola più probabile. Varianti più sofisticate, come il *beam search* o il *top-k sampling*, migliorano la diversità delle frasi generate.

5.7 Riepilogo

In questo capitolo abbiamo:

- compreso la struttura interna delle reti LSTM e il loro meccanismo di memoria;
- visto come vengono impiegate per generare frasi parola per parola a partire dalle feature visive;
- introdotto le tecniche di teacher forcing e scheduled sampling;
- descritto la funzione di perdita e la procedura di decodifica.

Riferimento al codice

Nel repository finale, la parte LSTM è integrata nel file `src/model.py`, che contiene sia la definizione del decoder sia le funzioni di generazione autoregressiva impiegate in validazione e test.

Capitolo 6

Addestramento del Modello CNN–LSTM

6.1 Integrazione di CNN e LSTM

Dopo aver descritto separatamente i due componenti principali del sistema — la rete convoluzionale (CNN) per l'estrazione delle feature e la rete LSTM per la generazione testuale — è ora possibile integrarli in un unico modello end-to-end.

L'obiettivo del modello CNN–LSTM è quello di mappare un'immagine in una sequenza di parole che ne descriva il contenuto. La pipeline può essere schematizzata come segue:

Immagine $\xrightarrow{\text{CNN}}$ Feature vector $\xrightarrow{\text{LSTM}}$ Didascalia generata

Idea chiave

La CNN funge da *encoder visivo*, comprimendo l'immagine in un vettore di caratteristiche, mentre la LSTM funge da *decoder linguistico*, traducendo quel vettore in parole.

6.2 Architettura combinata

La classe principale del modello CNN–LSTM unisce i due moduli e gestisce il flusso dei dati:

Listing 6.1: Architettura CNN–LSTM completa.

```
1 import torch
2 import torch.nn as nn
3
4 class EncoderDecoder(nn.Module):
5     def __init__(self, embed_dim, hidden_dim, vocab_size):
6         super(EncoderDecoder, self).__init__()
7         self.cnn = models.inception_v3(pretrained=True)
8         self.cnn = nn.Sequential(*list(self.cnn.children())[:-1]) #
          rimuove il classificatore
9         for param in self.cnn.parameters():
10             param.requires_grad = False
```

```

11
12     self.linear = nn.Linear(2048, embed_dim)
13     self.bn = nn.BatchNorm1d(embed_dim)
14     self.decoder = CaptionLSTM(embed_dim, hidden_dim, vocab_size)
15
16     def forward(self, images, captions):
17         with torch.no_grad():
18             features = self.cnn(images).squeeze(-1).squeeze(-1)
19             features = self.bn(self.linear(features))
20             outputs = self.decoder(captions, features)
21         return outputs

```

Spiegazione del flusso dati

- Le immagini vengono passate alla CNN, che produce un vettore di 2048 dimensioni.
- Tale vettore è proiettato in uno spazio di embedding di dimensione ridotta (ad es. 512).
- La LSTM utilizza il vettore come contesto iniziale per generare la sequenza di parole.
- L'output è una matrice di dimensione (batch, seq_len, vocab_size).

6.3 Preparazione dei dati per l'addestramento

Ogni campione del dataset deve contenere:

- l'immagine preprocessata (ridimensionata e normalizzata);
- la sequenza di parole tokenizzata e convertita in indici numerici;
- una versione "shifted" della sequenza per il training della LSTM (input e target sfalsati di un passo).

Listing 6.2: Costruzione del dataset PyTorch.

```

1 class FlickrDataset(Dataset):
2     def __init__(self, img_dir, captions_dict, vocab, transform):
3         self.img_dir = img_dir
4         self.captions_dict = captions_dict
5         self.vocab = vocab
6         self.transform = transform
7         self.image_ids = list(captions_dict.keys())
8
9     def __getitem__(self, idx):
10         img_id = self.image_ids[idx]
11         img_path = os.path.join(self.img_dir, img_id)
12         image = Image.open(img_path).convert("RGB")
13         image = self.transform(image)

```

```

14
15         caption = random.choice(self.captions_dict[img_id])
16         tokens = [self.vocab["<bos>"]] + [self.vocab.get(w, self.vocab["
            <unk>"])]
17             for w in caption.split()] + [self.vocab["<eos>"]]
18         return image, torch.tensor(tokens)
19
20     def __len__(self):
21         return len(self.image_ids)

```

6.4 Training loop

L'addestramento segue il classico schema del deep learning: *forward pass* → *calcolo della loss* → *backpropagation* → *aggiornamento dei pesi*.

Listing 6.3: Training loop per il modello CNN-LSTM.

```

1  for epoch in range(num_epochs):
2      model.train()
3      total_loss = 0
4      for images, captions in train_loader:
5          images, captions = images.to(device), captions.to(device)
6
7          outputs = model(images, captions[:, :-1]) # input
8          loss = criterion(outputs.reshape(-1, vocab_size), captions[:,
                1:].reshape(-1))
9
10         optimizer.zero_grad()
11         loss.backward()
12         nn.utils.clip_grad_norm_(model.parameters(), 1.0)
13         optimizer.step()
14         total_loss += loss.item()
15
16     avg_loss = total_loss / len(train_loader)
17     print(f"Epoch_{epoch+1}/{num_epochs}, Loss:_{avg_loss:.4f}")

```

Note importanti

- `captions[:, :-1]` è l'input alla LSTM, mentre `captions[:, 1:]` è il target.
- Si applica il **gradient clipping** (norma max = 1.0) per evitare esplosioni del gradiente.
- La CNN è congelata, ma la LSTM e il layer lineare vengono addestrati normalmente.

6.5 Validazione e metriche di valutazione

Durante o dopo l'addestramento, si valuta il modello sul set di test per misurare la qualità delle didascalie generate. La metrica più utilizzata è il **BLEU score** (Bilingual Evaluation Understudy), che misura la sovrapposizione di n-grammi tra la caption generata e quelle di riferimento.

Listing 6.4: Calcolo del punteggio BLEU.

```
1 from nltk.translate.bleu_score import corpus_bleu
2
3 references = [[ref.split() for ref in ref_list] for ref_list in
4               all_references]
5
6 hypotheses = [pred.split() for pred in generated_captions]
7
8 bleu1 = corpus_bleu(references, hypotheses, weights=(1.0, 0, 0, 0))
9 bleu4 = corpus_bleu(references, hypotheses, weights=(0.25, 0.25, 0.25,
10              0.25))
11
12 print(f"BLEU-1: {bleu1:.3f}, BLEU-4: {bleu4:.3f}")
```

Interpretazione delle metriche

- **BLEU-1** valuta la correttezza delle singole parole.
- **BLEU-4** misura la coerenza di frasi complete (4-grammi).
- Su Flickr8k, un punteggio BLEU-4 compreso tra 0.25 e 0.30 è considerato un buon risultato.

6.6 Salvataggio e caricamento dei modelli

Durante l'addestramento, è buona pratica salvare periodicamente i pesi del modello e poterli ricaricare per ulteriori sessioni.

Listing 6.5: Salvataggio e caricamento dei pesi.

```
1 torch.save(model.state_dict(), "checkpoints/model_epoch10.pt")
2
3 model.load_state_dict(torch.load("checkpoints/model_epoch10.pt"))
4 model.eval()
```

Buone pratiche

- Salvare i modelli migliori in base alla perdita di validazione.
- Annotare learning rate, batch size e seed random per riproducibilità.
- Tenere traccia dei risultati tramite file di log e riepiloghi metrici salvati.

6.7 Riepilogo

In questo capitolo abbiamo:

- integrato CNN e LSTM in un unico modello per il captioning;
- impostato il ciclo di addestramento e validazione;
- calcolato metriche di qualità (BLEU);
- introdotto strategie di stabilizzazione (gradient clipping, salvataggio checkpoint).

Riferimento al codice

Il codice completo per l'addestramento del modello CNN-LSTM è disponibile nel file `src/train.py` del repository GitHub del progetto.

Capitolo 7

Valutazione, Risultati e Discussione

7.1 Obiettivi della valutazione

Una volta completato l'addestramento, la valutazione del modello costituisce il passaggio necessario per verificare se l'architettura appresa riesca effettivamente a trasferire il contenuto visivo nello spazio linguistico in modo coerente. In un problema di *image captioning*, infatti, non è sufficiente osservare la diminuzione della funzione di perdita durante il training: è necessario stabilire se le didascalie prodotte risultino grammaticalmente plausibili, semanticamente pertinenti e sufficientemente specifiche rispetto all'immagine di input.

Nel contesto dell'*image captioning*, una buona valutazione combina:

- metriche quantitative (ad esempio BLEU, METEOR, ROUGE, CIDEr);
- analisi qualitative (osservazione diretta delle caption generate);
- esame dei log di training e validation per interpretare il comportamento dinamico dell'ottimizzazione.

Obiettivo della valutazione

Valutare un sistema di image captioning significa verificare che il modello sappia descrivere correttamente il contenuto dell'immagine in modo naturale e coerente, e non semplicemente “riconoscere” oggetti isolati.

7.2 Metriche quantitative

La metrica principale utilizzata nel progetto è il **BLEU score** (*Bilingual Evaluation Understudy*). Tale misura confronta la didascalia generata con le caption di riferimento umane, calcolando la sovrapposizione di n-grammi. Sebbene BLEU sia nato in ambito di traduzione automatica, esso rimane una delle metriche più usate anche nell'*image captioning*, in particolare quando si vuole confrontare in modo uniforme configurazioni diverse dello stesso modello.

$$BLEU_n = BP \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right)$$

dove:

- p_n è la precisione degli n-grammi;
- w_n sono i pesi assegnati a ciascun n-gramma (tipicamente uguali);
- BP è il *brevity penalty*, che penalizza caption troppo brevi.

Nel nostro caso, per ogni immagine vengono considerate cinque didascalie di riferimento, come previsto dal dataset Flickr8k. Questa caratteristica è particolarmente utile perché riduce l'effetto penalizzante che una singola formulazione linguistica potrebbe avere in un compito in cui più descrizioni differenti possono essere tutte corrette. Il calcolo del BLEU viene effettuato al termine della generazione delle caption sul set di test, così da confrontare tutte le configurazioni sullo stesso protocollo valutativo.

Interpretazione del BLEU

- BLEU-1 valuta la correttezza delle singole parole ($n=1$).
- BLEU-2 e BLEU-3 misurano coerenza parziale di brevi frasi.
- BLEU-4 cattura la struttura linguistica completa ($n=4$).

Nel contesto di un progetto accademico su Flickr8k, il BLEU va interpretato soprattutto in modo comparativo tra configurazioni omogenee dello stesso modello, più che come valore assoluto isolato.

7.3 Risultati sperimentali

La campagna sperimentale finale è stata organizzata come confronto controllato tra quattro configurazioni, tutte addestrate per 5 epoche in ambiente CPU-only. La baseline utilizza scheduled sampling e label smoothing; le due ablation successive rimuovono singolarmente queste tecniche, mentre l'ultima configurazione attiva il fine-tuning dell'encoder convoluzionale. In questo modo è possibile isolare il contributo delle diverse scelte metodologiche e valutarne l'impatto non solo sulla loss, ma soprattutto sulla qualità effettiva delle didascalie generate.

La tabella seguente riassume i risultati realmente ottenuti e salvati nei file `results/metrics_summary.csv` e `results/experiment_summary.csv` del repository.

Esperimento	BLEU-1	BLEU-2	BLEU-3	BLEU-4	Osservazioni
Baseline CPU	0.2553	0.1556	0.0928	0.0472	Configurazione di riferimento con scheduled sampling e label smoothing
No scheduled sampling	0.2232	0.1242	0.0717	0.0396	Peggiora la generalizzazione per mancanza di scheduled sampling
No label smoothing	0.2404	0.1435	0.0809	0.0394	Distribuzioni meno robuste per mancanza di label smoothing
Fine-tuning CNN	0.3301	0.2022	0.1189	0.0649	Miglior configurazione comparata

Tabella 7.1: Risultati comparativi reali ottenuti nelle quattro configurazioni principali del progetto.

Analisi dei risultati

I risultati mostrano un comportamento metodologicamente molto chiaro. La baseline costituisce un punto di partenza stabile, ma non rappresenta la configurazione migliore. La rimozione dello *scheduled sampling* e del *label smoothing* produce un peggioramento coerente su tutte le componenti BLEU, confermando che entrambe le tecniche contribuiscono positivamente alla generalizzazione del decoder. Il miglioramento più netto si osserva con il fine-tuning della CNN: l'adattamento dell'encoder visivo al dominio di Flickr8k porta il BLEU-4 da 0.0472 a 0.0649, suggerendo che una rappresentazione visiva più specializzata aiuta il decoder a generare caption più pertinenti.

L'aspetto più interessante emerso dalla tabella riguarda il rapporto tra loss e qualità finale delle caption. Le configurazioni *no scheduled sampling* e *no label smoothing* raggiungono infatti validation loss anche migliori della baseline, ma peggiorano sui punteggi BLEU. Questo comportamento mostra con chiarezza che, in un compito generativo come il captioning, una migliore ottimizzazione interna del modello non si traduce automaticamente in una migliore qualità linguistica esterna. In altre parole, il training può apparire più “facile” dal punto di vista della loss, pur portando a output meno robusti in inferenza.

7.4 Esempi di didascalie generate

La valutazione qualitativa integra quella quantitativa e consente di osservare in modo diretto il comportamento del modello. Gli esempi riportati di seguito provengono dagli artefatti realmente generati durante la fase di valutazione e sono stati selezionati per rappresentare tre casi distinti: una caption tipica della baseline, un caso qualitativamente riuscito della configurazione con fine-tuning e un *failure case* utile a discutere i limiti residui del sistema.

Predicted: man in the
Ground Truth: two ladies and three men looking at the ocean



Figura 7.1: Esempio 1: output rappresentativo della configurazione baseline.

Interpretazione. Nella baseline la didascalia risulta generalmente corretta sul piano gram-

maticale, ma tende a essere più generica e meno discriminativa. Questo comportamento è coerente con l'uso della CNN congelata, che fornisce feature visive robuste ma non ancora adattate in modo ottimale al dominio specifico del dataset. La frase prodotta, quindi, appare plausibile ma più vicina a uno schema descrittivo frequente che a una vera interpretazione fine della scena.

Predicted: brown and white dog is running in the grass
Ground Truth: brown dog running over grass



Figura 7.2: Esempio 2: output qualitativamente riuscito della configurazione con fine-tuning.

Interpretazione. Il fine-tuning dell'encoder migliora la specificità della descrizione: il modello coglie meglio il soggetto principale e l'azione dominante, producendo una caption più aderente al contenuto visivo. Questo riscontro qualitativo è pienamente coerente con il miglioramento quantitativo osservato su tutte le metriche BLEU e suggerisce che l'adattamento dell'estrattore visivo produca feature più informative per il decoder linguistico.

Predicted: hikes bread bread than war war so leading leading leading leading leading leading
leading leading leading leading leading leading leading leading
Ground Truth: four skiers walking up snow covered hill



Figura 7.3: Esempio 3: caso di fallimento utile a discutere i limiti del modello.

Interpretazione. Nel *failure case* il modello mostra rumore lessicale, ripetizioni o perdita di aderenza semantica. Questo tipo di errore evidenzia un limite tipico delle architetture ricorrenti in compiti di generazione: la tendenza a collassare su sequenze ad alta frequenza o a degradare quando la rappresentazione visiva non fornisce un segnale sufficientemente discriminante. L'inclusione di questo esempio è importante perché evita di costruire una valutazione qualitativa basata esclusivamente sui casi migliori.

Osservazione

L'analisi qualitativa completa quella quantitativa. Il modello è in grado di produrre caption spesso plausibili e sintatticamente corrette, ma la differenza tra baseline e fine-tuning emerge soprattutto nella precisione con cui viene selezionato il contenuto saliente dell'immagine. I casi di fallimento restano fondamentali, perché mostrano che il miglioramento numerico non elimina del tutto fenomeni di genericità, ripetizione o descrizione incompleta.

7.5 Grafici di training

Per monitorare l'apprendimento sono stati utilizzati i log di training e validation prodotti dal progetto. Nella versione finale del repository non sono stati conservati grafici esportati completi per tutte le run; di conseguenza, l'analisi seguente si basa sui valori realmente presenti nei log e nei riepiloghi degli esperimenti. Anche in assenza di una visualizzazione uniforme per tutte le configurazioni, i log permettono comunque di ricostruire il comportamento relativo delle diverse strategie di addestramento.

Interpretazione dei grafici

- La baseline termina con `final_train_loss = 4.8016` e `final_val_loss = 4.2332`, mentre la migliore validation loss della run è pari a 4.1927.
- Le ablation senza scheduled sampling e senza label smoothing mostrano loss di validazione anche migliori della baseline, ma ottengono BLEU inferiori: ciò conferma che la sola loss non è sufficiente a valutare la qualità di un modello generativo.
- Il fine-tuning della CNN porta al miglior BLEU-4 dell'intera campagna (**0.0649**), anche se non coincide con la loss finale più bassa. Questo dato è importante perché mostra una differenza concreta tra ottimizzazione del training e qualità testuale finale.

7.6 Discussione dei limiti

Nonostante il buon comportamento relativo della configurazione migliore, il modello CNN-LSTM presenta alcuni limiti strutturali che emergono con chiarezza sia dai punteggi quantitativi sia dagli esempi qualitativi:

- dipende fortemente dalla qualità delle caption di training;
- fatica a descrivere relazioni complesse tra oggetti (es. “il bambino accanto al cane”);

- genera frasi talvolta ripetitive o generiche (“a man standing on a street”).

Questi limiti non devono essere letti come un fallimento del progetto, ma come conseguenze prevedibili del setting sperimentale adottato. Il dataset è relativamente piccolo, il decoder è ricorrente e la generazione avviene senza tecniche di decoding più ricche come beam search. In questo quadro, la capacità del modello di produrre didascalie plausibili resta significativa, ma non elimina i problemi di specificità lessicale e di rappresentazione delle relazioni più sottili.

Cause e possibili soluzioni

- L’uso di un decoder **Transformer** potrebbe migliorare la modellazione del contesto linguistico.
- L’addestramento con una funzione di perdita basata su **CIDEr** o metodi di **Reinforcement Learning** permetterebbe di ottimizzare direttamente la qualità linguistica.
- L’ampliamento del dataset (es. MSCOCO) favorirebbe una migliore generalizzazione.

7.7 Conclusioni del capitolo

In sintesi, la valutazione del modello CNN–LSTM mostra che:

- il sistema è in grado di generare descrizioni plausibili e coerenti con il contenuto visivo;
- scheduled sampling e label smoothing contribuiscono in modo positivo alla generalizzazione del decoder;
- il fine-tuning della CNN rappresenta la variante più efficace nel setup sperimentale adottato.

Nel complesso, la sezione sperimentale conferma che il progetto non si limita a implementare una pipeline formalmente corretta, ma consente anche di osservare relazioni metodologicamente sensate tra scelte di training e qualità delle didascalie generate. In questo senso, il capitolo di valutazione costituisce uno dei punti più forti dell’elaborato, perché mostra in modo verificabile quali componenti abbiano effettivamente inciso sulle prestazioni finali.

Riferimento al codice

Gli script utilizzati per la valutazione e la generazione delle didascalie sono disponibili nei file:

- `src/eval.py` – calcolo delle metriche BLEU;
- `src/visualize.py` – generazione e salvataggio delle immagini con caption predetta;
- `logs/` e `results/` – log di training, riepiloghi numerici e predizioni salvate per ciascun esperimento.

Capitolo 8

Conclusioni e Prospettive Future

8.1 Sintesi del lavoro svolto

Il progetto ha avuto come obiettivo la realizzazione di un sistema completo di *image captioning* basato su architettura **CNN–LSTM**, capace di generare descrizioni testuali a partire da immagini e di essere valutato con un protocollo sperimentale esplicito. La pipeline sviluppata comprende la preparazione del dataset, la costruzione del vocabolario, l'implementazione del modello, l'addestramento con diverse configurazioni e l'analisi quantitativa e qualitativa dei risultati.

L'intero percorso ha attraversato tutte le fasi fondamentali di un progetto di deep learning:

- analisi del problema e definizione della pipeline *image-to-sequence*;
- preparazione e pulizia del dataset Flickr8k;
- estrazione delle feature visive tramite CNN pre-addestrata (InceptionV3);
- modellazione linguistica mediante LSTM e embedding trainabili;
- addestramento supervisionato con teacher forcing, scheduled sampling e label smoothing;
- valutazione quantitativa e qualitativa dei risultati ottenuti.

Risultato principale

La migliore configurazione sperimentale del progetto è risultata quella con fine-tuning della CNN, che ha raggiunto un punteggio **BLEU-4 pari a 0.0649**. Pur rimanendo lontano dalle prestazioni dei modelli più avanzati della letteratura, questo risultato conferma la coerenza della pipeline implementata e mostra un miglioramento netto rispetto alla baseline e alle ablation prive di scheduled sampling o label smoothing.

8.2 Contributi e aspetti innovativi

Tra gli aspetti qualificanti di questo lavoro si evidenziano:

- l'implementazione modulare in PyTorch, con codice leggibile e ampiamente commentato;

- l'uso di tecniche di stabilizzazione come gradient clipping e label smoothing;
- l'applicazione di **scheduled sampling** per migliorare la robustezza del decoder LSTM;
- l'utilizzo di 5 caption per immagine nella valutazione BLEU, rendendo la metrica più affidabile;
- la visualizzazione interattiva dei risultati tramite interfaccia Streamlit.

Nel complesso, questi elementi rendono il progetto riproducibile, facilmente ispezionabile e adatto a successive estensioni sperimentali. Il contributo più concreto non è tanto l'adozione di una singola tecnica, quanto la costruzione di una pipeline coerente in cui ciascuna scelta può essere verificata e discussa alla luce dei risultati ottenuti.

8.3 Limiti riscontrati

I risultati ottenuti mostrano anche con chiarezza alcuni limiti strutturali del modello CNN-LSTM adottato:

- la difficoltà nel catturare relazioni complesse tra oggetti e azioni;
- la tendenza a produrre frasi grammaticalmente corrette ma poco varie;
- la dipendenza da dataset relativamente piccoli come Flickr8k;
- la mancanza di attenzione visiva (il modello non “guarda” regioni specifiche dell'immagine).

Analisi critica

Questi limiti derivano in gran parte dalla natura sequenziale del decoder LSTM, dall'assenza di un meccanismo di attenzione visiva esplicito e dalla dimensione ridotta del dataset. In un contesto di progetto, tali vincoli non invalidano il lavoro svolto, ma definiscono con precisione l'ambito entro cui i risultati devono essere interpretati.

8.4 Prospettive future

Le estensioni più naturali del progetto riguardano sia l'architettura sia la strategia di valutazione:

- sostituzione del decoder LSTM con un **Transformer decoder**, in grado di gestire contesti più lunghi e paralleli;
- introduzione di un **meccanismo di attenzione visiva** per localizzare oggetti rilevanti nell'immagine;
- addestramento con metodi di **Reinforcement Learning** per ottimizzare metriche linguistiche (CIDEr, SPICE);
- espansione a dataset più ampi e complessi (MSCOCO, Flickr30k);

- sviluppo di un sistema multimodale in tempo reale integrabile in applicazioni di accessibilità visiva.

Confronto concettuale con modelli più recenti

L'architettura CNN-LSTM impiegata in questo progetto rappresenta una soluzione classica, oggi meno competitiva rispetto agli approcci basati su attenzione e Transformer, ma ancora estremamente utile per studiare in modo trasparente il legame tra rappresentazione visiva, modellazione sequenziale e qualità delle caption generate. In questo senso, il modello adottato costituisce una base metodologica solida da cui partire per confronti futuri con architetture più recenti.

8.5 Conclusione generale

Nel complesso, il lavoro svolto documenta un percorso completo di progettazione, sviluppo e valutazione di un sistema di *image captioning*. Il valore del progetto non risiede soltanto nell'implementazione dell'architettura CNN-LSTM, ma soprattutto nella possibilità di osservare in modo verificabile come diverse scelte di training influenzino la qualità finale delle descrizioni generate. Da questo punto di vista, la campagna sperimentale ha fornito un risultato chiaro: scheduled sampling, label smoothing e fine-tuning dell'encoder non sono dettagli marginali, ma componenti che incidono concretamente sul comportamento del modello.

Considerazione finale

Il progetto fornisce una base affidabile per ulteriori sviluppi nel captioning multimodale. Pur collocandosi entro un setting sperimentale contenuto, esso mostra come una pipeline ben costruita, correttamente documentata e supportata da risultati reali possa costituire un elaborato solido e tecnicamente difendibile.

Capitolo 9

Analisi dei file e delle funzioni implementate

In questa sezione viene fornita una descrizione dettagliata delle componenti software sviluppate per il progetto di *Image Captioning* basato su architettura CNN-LSTM. L'obiettivo è illustrare in modo chiaro le funzioni principali di ciascun file, spiegandone il ruolo, la logica interna e le interazioni con le altre parti del sistema. Tutti i file sono scritti in linguaggio Python e utilizzano il framework PyTorch per la modellazione neurale.

9.1 File `vocab_builder.py`

Questo modulo si occupa della costruzione del vocabolario testuale a partire dalle caption del dataset. Le funzioni principali sono:

- **`clean_caption(caption)`** – Pulisce una singola caption rendendola uniforme: converte il testo in minuscolo, rimuove punteggiatura, numeri e parole isolate, restituendo una stringa pulita pronta per la tokenizzazione.
- **`build_vocabulary(captions, min_freq)`** – Crea un dizionario di parole (vocabolo–indice) contando la frequenza di ogni termine. Solo le parole che appaiono almeno `min_freq` volte vengono incluse per ridurre la dimensione del vocabolario.
- **`save_vocabulary(vocab, path)`** – Salva il vocabolario in formato JSON per essere caricato successivamente durante il training.
- **`load_vocabulary(path)`** – Carica un vocabolario salvato in precedenza da file.

Questo file è il primo passo del processo, poiché produce il dizionario token–indice necessario per convertire le caption testuali in sequenze numeriche gestibili dalla rete LSTM.

9.2 File `dataset.py`

Definisce la classe `FlickrDataset`, che eredita da `torch.utils.data.Dataset` e gestisce l'associazione tra immagini e caption. Le funzioni principali sono:

- **__init__** – Inizializza il dataset, caricando le informazioni da un file CSV contenente i nomi delle immagini e le caption. Riceve anche il dizionario del vocabolario e le trasformazioni da applicare alle immagini.
- **__getitem__(index)** – Restituisce una coppia immagine-caption alla posizione specificata. L'immagine viene caricata e trasformata (resize, normalizzazione), mentre la caption viene convertita in una sequenza di indici, con l'aggiunta dei token speciali `<bos>` e `<eos>`.
- **__len__** – Restituisce la lunghezza del dataset, utile per il DataLoader PyTorch.

Questo modulo è il ponte tra i dati grezzi e la rete neurale, preparando batch coerenti di immagini e testo per il training.

9.3 File `model.py`

Contiene la definizione dell'architettura neurale CNN-LSTM e rappresenta il cuore del progetto. Le funzioni principali sono:

- **EncoderCNN** – Classe che utilizza una CNN pre-addestrata (InceptionV3 o ResNet50) per estrarre feature visive dall'immagine. Rimuove gli strati di classificazione finali e aggiunge una proiezione lineare per ridurre la dimensionalità delle feature.
- **DecoderLSTM** – Classe che definisce la rete ricorrente (LSTM) e un layer fully connected che mappa lo stato nascosto sull'intero vocabolario, producendo una distribuzione di probabilità sulle parole successive.
- **forward(image, caption)** – Descrive il passaggio in avanti: le feature estratte dalla CNN vengono fornite alla LSTM, che genera la sequenza testuale passo per passo.
- **sample(features)** – Metodo usato in fase di test per generare una caption, parola dopo parola, partendo solo dalle feature visive.

9.4 File `train.py`

Implementa la logica di addestramento del modello e la gestione delle epoche. Le funzioni principali includono:

- **train_one_epoch(model, dataloader, optimizer, criterion)** – Esegue un'epoca di addestramento: calcola la perdita, effettua il backpropagation e aggiorna i pesi.
- **evaluate(model, val_loader)** – Valuta il modello sul set di validazione, restituendo la perdita media e (facoltativamente) la metrica BLEU.
- **save_checkpoint(state, path)** – Salva lo stato corrente del modello (pesi, epoca, ottimizzatore).
- **train()** – Funzione principale che gestisce il ciclo di addestramento completo, l'aggiornamento del learning rate, il salvataggio dei migliori pesi e l'applicazione dello *scheduled sampling*.

9.5 File `eval.py`

Questo script esegue la valutazione del modello addestrato e genera caption per le immagini di test. Le funzioni chiave sono:

- **`generate_caption(model, image)`** – Utilizza il metodo `sample()` del decoder per produrre una frase descrittiva a partire da un'immagine.
- **`compute_bleu(references, predictions)`** – Calcola la metrica BLEU confrontando le caption predette con quelle reali, utilizzando tutte le 5 descrizioni disponibili per ogni immagine.
- **`evaluate_model(model, test_loader)`** – Esegue la valutazione completa sull'intero dataset di test, restituendo il punteggio BLEU complessivo e salvando i risultati.

9.6 File `visualize.py`

Si occupa di mostrare esempi qualitativi di caption generate. Carica un sottoinsieme di immagini, genera le rispettive descrizioni e le salva in un'unica figura comparativa (immagine + caption reale + caption predetta). Include inoltre una funzione per creare overlay grafici direttamente sull'immagine.

9.7 File `ui_streamlit.py`

Contiene una semplice interfaccia utente sviluppata con Streamlit, che consente di caricare un'immagine e ottenere in tempo reale la didascalia generata dal modello. Le funzioni principali sono:

- **`load_model(weights_path)`** – Carica i pesi del modello addestrato.
- **`predict_caption(image)`** – Esegue la generazione della didascalia a partire da un'immagine fornita dall'utente.
- **`main()`** – Crea l'interfaccia grafica, gestendo il caricamento delle immagini e la visualizzazione dei risultati.

9.8 File `utils.py`

Contiene funzioni di supporto comuni a più moduli:

- **`set_seed(seed)`** – Imposta un seme casuale per rendere gli esperimenti riproducibili.
- **`save_config(config)`** – Salva le configurazioni di training in un file YAML.
- **`count_parameters(model)`** – Restituisce il numero totale di parametri addestrabili del modello.

- **decode_tokens(tokens, vocab)** – Converte una sequenza di indici numerici nella corrispondente sequenza di parole.

Insieme, questi moduli costituiscono un sistema completo e modulare per l'addestramento, la valutazione e l'utilizzo di un modello di *image captioning* basato su CNN-LSTM. Ogni file ha un ruolo specifico e interagisce con gli altri tramite interfacce chiare e ben definite, in modo da garantire manutenibilità e riutilizzabilità del codice.

Appendice A

Struttura del Repository e Istruzioni d'Uso

A.1 Repository GitHub del progetto

Il codice sorgente completo, i file di configurazione e gli artefatti di addestramento sono disponibili nel repository ufficiale del progetto:

Repository ufficiale

<https://github.com/giuseppedimonte/image-captioning-cnn-lstm>

Il repository è pubblico e contiene tutto il materiale necessario per eseguire, testare e valutare il modello. È strutturato in modo modulare per facilitare la comprensione, la riproducibilità e l'estensione del lavoro.

A.2 Struttura delle cartelle

```
image-captioning-cnn-lstm/
|
|-- src/                                # Codice sorgente principale
|   |-- dataset.py                      # Gestione del dataset e preprocessing
|   |-- model.py                        # Definizione del modello CNN-LSTM
|   |-- train.py                        # Script di addestramento
|   |-- eval.py                         # Script di valutazione (BLEU)
|   |-- visualize.py                   # Generazione e visualizzazione caption
|   |-- vocab_builder.py               # Creazione del dizionario token-indice
|   \-- utils.py                       # Funzioni di supporto
|
|-- configs/                            # File di configurazione
|   |-- config.yml                     # Configurazione generale del progetto
|   |-- baseline_cpu.yml
|   |-- ablation_no_scheduled_sampling_cpu.yml
```

```

| |-- ablation_no_label_smoothing_cpu.yml
| \-- ablation_finetune_cpu.yml
|
|-- checkpoints/                # Pesi salvati dei modelli
| |-- baseline_cpu/
| |-- ablation_no_ss_cpu/
| |-- ablation_no_ls_cpu/
| \-- ablation_finetune_cpu/
|
|-- logs/                        # Log testuali di training ed evaluation
|-- results/                    # Predizioni e riepiloghi metrici
|
|-- figures/                    # Immagini e grafici per il report
|
|-- requirements.txt            # Dipendenze Python
|-- README.md                  # Documentazione del progetto
\-- LICENSE                    # Licenza open-source (MIT)

```

Nota

La struttura è progettata per rispettare le buone pratiche di sviluppo in PyTorch e per garantire la riproducibilità dei risultati.

A.3 Installazione e requisiti

A.3.1 Requisiti di sistema

- Python 3.10 o superiore
- PyTorch ≥ 2.0 con supporto CUDA (GPU opzionale ma consigliata)
- NLTK, NumPy, Pillow, Matplotlib, PyYAML

A.3.2 Installazione passo per passo

```

# Clonare il repository
git clone https://github.com/giuseppedimonte/image-captioning-cnn-lstm.git
cd image-captioning-cnn-lstm

# Creare un ambiente virtuale
python -m venv venv
source venv/bin/activate    # (Linux/Mac)
venv\Scripts\activate      # (Windows)

# Installare le dipendenze
pip install -r requirements.txt

```

Suggerimento

Per eseguire l'addestramento su GPU è consigliato verificare la corretta installazione di `torch.cuda.is_available()`.

A.4 Esecuzione del training

L'addestramento del modello può essere lanciato direttamente tramite terminale. Nel protocollo sperimentale finale, la configurazione di riferimento utilizzata è stata la baseline CPU:

```
python src/train.py --config configs/baseline_cpu.yml
```

Durante l'addestramento, verranno generati:

- log testuali in `logs/`;
- checkpoint nelle sottocartelle di `checkpoints/`;
- predizioni e riepiloghi metrici in `results/`.

Esempio di configurazione YAML

```
train:
  batch_size: 16
  epochs: 5
  learning_rate: 0.0005
  teacher_forcing_start: 0.7
  teacher_forcing_end: 0.3
  label_smoothing: 0.1
model:
  embed_dim: 256
  hidden_dim: 512
  dropout: 0.3
```

A.5 Valutazione e generazione caption

Per valutare il modello addestrato e generare nuove caption:

```
python src/eval.py --images_dir data/Flickr8k_Dataset \
  --captions_json data/Flickr8k_text/test_captions.json \
  --vocab_path data/vocab.json \
  --weights checkpoints/baseline_cpu/model_best.pt
```

Le didascalie prodotte verranno salvate in formato JSON o CSV e potranno essere visualizzate con:

```
python src/visualize.py --weights checkpoints/baseline_cpu/model_best.pt \
  --vocab_path data/vocab.json \
  --images_dir data/Flickr8k_Dataset \
  --captions_json data/Flickr8k_text/test_captions.json \
  --output_dir figures/baseline_cpu
```

A.6 Esecuzione dell'interfaccia Streamlit

L'interfaccia utente, sviluppata con Streamlit, permette di caricare un'immagine e visualizzare la caption generata in tempo reale:

```
streamlit run src/ui_streamlit.py
```

Esempio d'uso

Caricare un file immagine dal browser, attendere l'elaborazione e visualizzare la didascalia proposta dal modello. L'interfaccia permette di verificare in tempo reale il comportamento del modello su nuove immagini, costituendo un supporto qualitativo utile accanto alla valutazione quantitativa.

A.7 Riproducibilità dei risultati

Per garantire risultati ripetibili:

- impostare il seme casuale globale (`torch.manual_seed(42)`);
- salvare il file di configurazione utilizzato;
- annotare il numero di epoche, la GPU usata e la versione di PyTorch.

Buone pratiche finali

Il repository è organizzato secondo principi di riproducibilità e tracciabilità. Tutti i risultati principali (punteggi BLEU, immagini qualitative, log di training ed evaluation, predizioni JSON) sono documentati e archiviati per verifica.

Appendice B – Glossario dei Termini Tecnici

Introduzione

Questa appendice raccoglie e spiega in modo sintetico ma preciso i principali concetti e termini tecnici utilizzati nel progetto. Il glossario ha lo scopo di fornire una visione chiara e unificata delle componenti che concorrono al funzionamento del sistema di *image captioning* basato su CNN-LSTM.

Addestramento (Training) Processo mediante il quale una rete neurale apprende dai dati, ottimizzando i propri parametri interni (pesi e bias) per ridurre l'errore tra le predizioni e i valori attesi.

Batch Insieme di campioni elaborati contemporaneamente durante un passo dell'addestramento. L'uso dei batch migliora la stabilità e l'efficienza computazionale del training.

BLEU (Bilingual Evaluation Understudy) Metrica di valutazione per la generazione di testo. Misura la sovrapposizione di sequenze di parole (*n-grammi*) tra la frase generata e quelle di riferimento, penalizzando caption troppo corte.

CNN (Convolutional Neural Network) Architettura neurale specializzata nell'elaborazione di immagini. Utilizza operazioni di convoluzione per rilevare pattern locali come bordi, texture e forme.

Caption Descrizione testuale che sintetizza il contenuto di un'immagine. Nel contesto del progetto, è la sequenza di parole generata dal modello a partire da un input visivo.

Dataset Insieme di dati (immagini e descrizioni) utilizzato per addestrare e valutare il modello. In questo progetto viene impiegato il dataset **Flickr8k**, che contiene 8.000 immagini e 5 descrizioni per ciascuna.

Embedding Rappresentazione vettoriale densa di parole o immagini. Nel caso del linguaggio naturale, ogni parola è associata a un vettore numerico che cattura relazioni semantiche e sintattiche.

Epoca (Epoch) Un'epoca corrisponde a un passaggio completo del modello su tutto il dataset di addestramento. L'apprendimento avviene gradualmente su più epoche, fino a convergere.

Feature (Caratteristica) Informazione numerica estratta da un'immagine (es. presenza di forme, colori, texture). Le feature sono la base su cui la LSTM genera le descrizioni.

Fine-tuning Tecnica che consiste nell'adattare un modello pre-addestrato (ad esempio una CNN) a un nuovo compito, modificando solo gli ultimi strati. Permette di ridurre i tempi di addestramento e migliorare le prestazioni su dataset piccoli.

Gradient Clipping Tecnica di stabilizzazione che limita l'ampiezza dei gradienti durante la retropropagazione. Serve a evitare che valori troppo grandi causino instabilità numerica o blocchi nel training.

Hidden State (Stato Nascosto) Rappresentazione interna mantenuta dalla LSTM a ogni passo temporale. Contiene informazioni sul contesto linguistico accumulato fino a quel punto della sequenza.

LSTM (Long Short-Term Memory) Variante delle reti ricorrenti (RNN) capace di modellare dipendenze a lungo termine grazie all'uso di tre porte: input, forget e output. È il cuore della generazione di testo nel modello proposto.

Loss Function (Funzione di Perdita) Misura la differenza tra l'output del modello e il valore corretto. Nel progetto è utilizzata la *Cross Entropy Loss*, comune nei problemi di classificazione.

Overfitting Fenomeno per cui un modello impara eccessivamente i dati di training, perdendo capacità di generalizzazione. Si controlla con tecniche come dropout, early stopping e data augmentation.

Scheduled Sampling Strategia che riduce progressivamente il *teacher forcing*, abituando il modello a usare le proprie predizioni durante l'addestramento. Migliora la robustezza e la naturalezza delle frasi generate.

Teacher Forcing Tecnica in cui, durante il training, la parola corretta (ground truth) viene fornita come input successivo alla LSTM, invece della parola predetta. Accelerando l'apprendimento ma riducendo la robustezza al test.

Token Unità linguistica elementare (parola o simbolo) in cui viene scomposto un testo. I token vengono numericamente codificati tramite un dizionario (*vocab*).

Transfer Learning Metodo che sfrutta le conoscenze di un modello pre-addestrato su un grande dataset per un nuovo compito. In questo progetto, la CNN (InceptionV3) è stata usata come estrattore di feature pre-addestrato su ImageNet.

Transformer Architettura neurale basata su meccanismi di attenzione, oggi dominante nel campo dell'IA multimodale. È considerata l'evoluzione naturale delle RNN e LSTM, capace di elaborare sequenze in parallelo.

Questo glossario intende offrire una visione d'insieme chiara e rigorosa dei concetti fondamentali trattati nel progetto, fungendo da supporto alla comprensione dei capitoli tecnici.

Appendice C – Timeline del Progetto e Riproducibilità degli Esperimenti

Introduzione

Questa appendice documenta il percorso di sviluppo del progetto, illustrando le principali fasi, le attività svolte in ciascun momento e le procedure adottate per garantire la riproducibilità completa degli esperimenti.

A.8 Timeline del progetto

La seguente tabella riassume le tappe fondamentali del progetto, dalla fase iniziale di analisi alla presentazione finale.

Fase	Descrizione	Attività principali
Analisi iniziale	Comprensione del problema	Studio del dataset Flickr8k, revisione della letteratura sui modelli CNN–LSTM, definizione degli obiettivi del progetto.
Progettazione	Definizione dell’architettura	Pianificazione della pipeline image-to-sequence, scelta dei parametri principali e struttura del repository.
Implementazione	Sviluppo del codice	Creazione del dataloader, preprocessing del testo, costruzione del vocabolario e definizione del modello CNN–LSTM in PyTorch.
Addestramento	Ottimizzazione dei parametri	Esecuzione degli esperimenti, tuning degli iperparametri, monitoraggio della perdita e salvataggio dei checkpoint.
Valutazione	Analisi delle prestazioni	Calcolo delle metriche BLEU, analisi qualitativa delle caption e confronto tra diverse configurazioni.
Visualizzazione	Presentazione dei risultati	Creazione di grafici, immagini esemplificative e sviluppo dell’interfaccia Streamlit per la generazione in tempo reale.
Documentazione finale	Sintesi e revisione	Stesura del report accademico, commento del codice e preparazione della discussione orale.

Tabella A.1: Fasi principali del progetto e relative attività.

A.9 Controllo di versione e tracciabilità

Per garantire trasparenza e riproducibilità, il codice è stato gestito tramite **Git** e pubblicato su GitHub con commit regolari. Ogni commit corrisponde a una modifica coerente (nuova funzionalità, correzione, esperimento o aggiornamento documentale) ed è accompagnato da un messaggio standardizzato:

```
feat: add scheduled sampling in training loop
fix: correct BLEU score computation in eval.py
docs: update README with installation instructions
```

Inoltre, ogni esperimento è stato eseguito utilizzando un file di configurazione YAML dedicato, contenente tutti i parametri di training (learning rate, batch size, numero di epoche, dropout, ecc.), archiviato nella directory `configs/`.

A.10 Riproducibilità degli esperimenti

Per replicare gli esperimenti descritti nel documento:

1. Clonare il repository del progetto.

2. Installare le dipendenze elencate in `requirements.txt`.
3. Scaricare e posizionare il dataset Flickr8k nella cartella `data/`.
4. Avviare l'addestramento con:

```
python src/train.py --config configs/baseline_cpu.yml
```

5. Valutare il modello addestrato:

```
python src/eval.py --images_dir data/Flickr8k_Dataset --captions_json data/Flickr8k_t
```

6. Visualizzare le caption generate:

```
python src/visualize.py --weights checkpoints/baseline_cpu/model_best.pt --vocab_path
```

A.11 Suggerimenti pratici per la replica

- Utilizzare un ambiente virtuale o `conda` per evitare conflitti di versione.
- Impostare un seme casuale fisso per garantire la comparabilità dei risultati.
- Salvare log testuali e riepiloghi metrici per ogni esperimento.
- Eseguire test con diversi learning rate (es. 0.001, 0.0005, 0.0001) e documentare le differenze di prestazione.
- Mantenere una cartella separata per ciascun esperimento (`exp1`, `exp2`, ecc.) con i relativi file di configurazione e pesi.

A.12 Conclusione

Il processo di sviluppo e validazione del modello è stato pianificato e tracciato con attenzione, in modo da garantire che ogni risultato riportato sia verificabile e riproducibile. Questa appendice fornisce le informazioni necessarie per replicare integralmente gli esperimenti e proseguire con ulteriori miglioramenti o confronti futuri.